

Clase 9 · Refinando el Backlog

INVEST, Criterios de Aceptación, Priorización y Estimación

Materia: **Modelación y Diseño de Sistemas 2026** — UNC · ESCMB · AUSI

Docente: **Ing. Fernando Bono**

90 MIN

CLASE 9

MyD 2026 · Ing. Fernando Bono · Clase 9

Agenda de la clase

01

Repaso Clase 8 (5 min)

02

Por qué importa la calidad de las HU (5 min)

03

INVEST · los 6 criterios uno por uno (20 min)

04

Criterios de Aceptación · G/W/T y checklist (10 min)

05

Cobertura mínima y antipatrones de CA (5 min)

06

Priorización · MoSCoW, WSJF, Valor-Esfuerzo (10 min)

07

Estimación · Story Points y Planning Poker (10 min)

08

Velocity y Capacity (5 min)

09

Ejercicio en clase (20 min)

Repaso · lo que vimos en Clase 8

De OAS al backlog · Jerarquía Epic / Feature / HU / Task.

Nivel	Qué es	Tamaño
📦 **EPIC**	Gran bloque de valor	Multi-sprint (R1/R2)
🎯 **FEATURE**	Capacidad concreta del sistema	1-3 sprints
📘 **USER STORY**	Unidad mínima de valor para un rol	≤ 1 sprint (≤8 SP)
🔧 **TASK**	Paso técnico para implementar la HU	Horas

ⓘ Hoy: cómo sabemos que una HU está **bien escrita** y cómo la estimamos.

¿Por qué importa la calidad de las HU?

> **Una HU mal escrita es un bug esperando suceder.**

Síntomas de HU de mala calidad

- ❌ El equipo debate una semana cuánto estima
- ❌ QA no sabe qué testear
- ❌ El PO rechaza el resultado ("no era esto")
- ❌ La HU explota y genera 3 bugs en producción
- ❌ Se posterga sprint tras sprint sin poder arrancar

Una HU bien escrita

- ✅ Se estima en minutos
- ✅ QA escribe el test desde el CA
- ✅ El PO reconoce el resultado al verlo

INVEST · Seis criterios de calidad

Acrónimo propuesto por **Bill Wake (2003)** — checklist para toda HU antes del sprint.

I

Independent

N

Negotiable

V

Valuable

E

Estimable

S

Small

T

Testable

Vamos uno por uno con ejemplos reales del caso **ChangoMás**.

I · Independent (Independiente)

> **La HU se puede implementar sin esperar a otra.**

Regla: minimizar dependencias entre HU del mismo sprint.

¿Por qué importa?

- Si HU-A depende de HU-B, no podés paralelizar
- Bloqueos generan tiempo muerto en el sprint
- Dificulta reordenar el backlog

Ejemplo ChangoMás

❌ **Mal:** "HU-B: Pagar con tarjeta" depende de "HU-A: Catálogo de productos"

✅ **Bien:** HU-015 Pagar con tarjeta es independiente — puede testearse con un carrito mock

Cuando la dependencia es inevitable: ordenar el backlog para que el orden físico la respete.

N · Negotiable (Negociable)

> **La HU describe el QUÉ, no el CÓMO. El cómo se conversa con el equipo.**

Regla: la HU es el punto de partida de una conversación, no un contrato cerrado.

¿Qué NO debe tener la HU?

- ❌ Tecnologías específicas ("usar Redis")
- ❌ Decisiones de arquitectura ("implementar con microservicio")
- ❌ Detalles de UI ("botón azul en la esquina")

Ejemplo ChangoMás

❌ **Mal:** "Como cliente quiero pagar con tarjeta usando la API de Prisma v3"

✅ **Bien:** "Como cliente quiero pagar con tarjeta para cerrar mi compra sin efectivo"

El "con qué API" se decide en refinement técnico, no en la HU.

V · Valuable (Valiosa)

> **Aporta valor observable para el usuario o el negocio.**

Regla: si no podés completar la frase "para que…" con un beneficio real, no es una HU.

❌ ¿Qué NO es valor?

- "Para cumplir el requerimiento" (tautológico)
- "Para que esté implementado" (circular)
- "Para refactorizar" (técnico, sin valor de negocio)

✅ Ejemplo ChangoMás

❌ **Mal:** "Para tener ticket en base de datos"

✅ **Bien:** "Para que el cliente pueda demostrar su compra en control de seguridad"

📊 El valor se mide en KPIs

- Tiempo ahorrado
- Errores reducidos
- Conversión
- Satisfacción del cliente

E · Estimable

> **El equipo puede asignar Story Points con confianza razonable.**

¿Por qué no puedo estimar una HU?

- 😕 No entiendo el alcance funcional
- 😕 No sé qué tecnología usar
- 😕 Hay demasiados unknowns técnicos
- 😕 La HU es tan grande que no sé cuánto durará

Soluciones

- Falta info funcional → preguntar al PO
- Falta claridad técnica → **spike de investigación** (timeboxed, 2-4h)
- Es demasiado grande → partirla

⚠ **Si la HU lleva más de 5 minutos estimarla, probablemente violó Estimable.**

S · Small (Pequeña)

> **Debe caber en un sprint · lo ideal es ≤ 8 Story Points.**

Escala Fibonacci: 1, 2, 3, 5, 8, 13, 21, 34…

SP	Interpretación
1-3	HU chica, ideal
5	HU mediana, cómoda
8	HU grande — aceptable pero en el límite
13	**Partirla** — hay incertidumbre o alcance amplio
21+	**Es un Feature disfrazado** — descomponer

Ejemplo ChangoMás: HU-011 Escanear código de barras → 3 SP ✅ · HU-021 Detectar discrepancia báscula/ticket tiempo real → 8 SP · en el límite, candidata a partir

T · Testable

> **Existe una condición objetiva que dice "esta HU está terminada".**

Prueba de fuego: si QA no puede escribir un test automatizado desde el CA, el CA está mal escrito.



Criterios verificables

No subjetivos — deben poder comprobarse objetivamente



Inputs y outputs concretos

Valores específicos, no descripciones vagas



Casos de error explícitos

El comportamiento ante fallas debe estar definido

❌ **Mal:** "El escaneo debe ser rápido"

✅ **Bien:** "La respuesta visual al cliente ocurre en <1 segundo tras escanear"

INVEST · Checklist consolidado

Pasá por este filtro toda HU antes del sprint.

Criterio	Pregunta clave
Independent	¿Se puede implementar sin esperar otra HU?
Negotiable	¿Describe el qué, no el cómo?
Valuable	¿El "para…" describe valor real y medible?
Estimable	¿Puedo asignarle SP con confianza?
Small	¿Entra en un sprint? ≤ 8 SP
Testable	¿Los CA son verificables objetivamente?

⚠ **Si una HU falla algún criterio: refinarla antes de meterla al sprint.**

Antipatrones frecuentes · ChangoMás

Ejemplos reales mal escritos vs bien escritos.

❌ Mal	✅ Bien
"Como usuario quiero usar el sistema"	HU-011: Como cliente quiero escanear un producto para agregarlo al carrito
"Implementar self-checkout"	HU-001 + HU-011 + HU-015 + HU-017 (4 HU distintas)
"Como dev quiero usar Redis para el catálogo"	HU funcional del catálogo · Redis se decide en refinement técnico
"Hacer que la caja funcione"	HU-011 (escanear) + HU-012 (código manual) + HU-013 (quitar)

Patrón general: **rol del negocio + verbo específico + beneficio observable.**

Criterios de Aceptación

> **El contrato con QA · el Definition of Done al nivel de la HU.**

> "Si el equipo de QA no puede escribir un test automatizado desde el CA, el CA está mal escrito."



Verificables

Condiciones que deben cumplirse para que la HU esté Done



Del lado del negocio

No de la implementación técnica



Independientes

Cada CA es autónomo entre sí



Observables

El PO puede verificarlos sin mirar código

Dos formatos recomendados: **Given/When/Then** y **lista de verificación**.

Formato Given / When / Then (BDD)

Estándar BDD · compatible con Gherkin · traducible directamente a tests automatizados.

```
GIVEN / DADO
   <contexto o precondición>

WHEN / CUANDO
   <acción o evento disparador>

THEN / ENTONCES
   <resultado esperado verificable>
```

¿Cuándo usarlo?

Útil cuando el comportamiento depende de un **contexto** y hay un **evento** claro.

Herramientas compatibles

- Cucumber
- SpecFlow
- Gherkin
- Playwright (BDD)

Ejemplo G/W/T · HU-011 ChangoMás

HU-011 · Como cliente quiero escanear el código de barras de un producto para agregarlo automáticamente al carrito.

CA-01 · Happy Path 🟢

```
DADO que la caja está disponible
  y el producto tiene código de barras válido
CUANDO el cliente pasa el producto
  por el lector
ENTONCES el producto se agrega al carrito
  con cantidad=1 y precio unitario
Y el monto total del carrito se actualiza.
```

CA-03 · Error 🔴

```
DADO un código que no existe
  en el catálogo
CUANDO el cliente lo escanea
ENTONCES el sistema muestra
  "Producto no encontrado,
   solicite asistencia"
Y activa alerta visual al supervisor
Y NO agrega el producto al carrito.
```

Formato alternativo · Lista de verificación

Cuándo usar G/W/T y cuándo usar checklist.

G/W/T — cuándo usarlo

- ✅ El comportamiento depende del contexto
- ✅ Hay un evento disparador claro
- ✅ Se va a automatizar con Cucumber/SpecFlow
- ✅ Hay múltiples ramas (happy + error + borde)

Lista de verificación — cuándo usarla

- ✅ Los CA son declarativos ("el ticket incluye X, Y, Z")
- ✅ No hay evento disparador claro
- ✅ Son validaciones de formato o contenido
- ✅ Ejemplo: campos de un formulario, elementos de un ticket

Cobertura mínima de CA

Al menos 1 CA por cada tipo — no alcanza con el feliz camino.

Tipo	Qué cubre	Ejemplo HU-011
🟢 **Happy path**	Flujo principal exitoso	CA-01 Escaneo exitoso
🟡 **Alternativo**	Vía alterna válida	CA-02 Código no legible → ingreso manual
🔴 **Error / excepción**	Falla esperada y su respuesta	CA-03 Código no existe en catálogo
⚪ **Borde / límite**	Valor límite o condición especial	CA-05 Producto restringido → bloqueo
⚡ **Performance**	RNF aplicable a la HU	CA-04 Respuesta <1s

⚠ **Regla:** si tu HU tiene solo happy path, le falta cobertura.

Antipatrones de Criterios de Aceptación

Los tres pecados capitales · evitarlos a toda costa.

1

Vagos

❌ "El sistema debe ser rápido" — No es verificable, ¿rápido es cuánto?

✅ **Bien:** "La respuesta al escaneo ocurre en <1 segundo"

2

Técnicos

❌ "Implementar usando Redis con cache TTL 300s" — Es decisión técnica, no CA

✅ **Bien:** "El catálogo responde en <500ms a consultas repetidas"

3

Tautológicos

❌ "Cumplir con los RF del sistema" — No dice nada concreto

✅ **Bien:** CA específico por cada RF relevante a la HU

Priorización · no todas las HU son iguales

> **El Product Backlog es una lista ordenada, no un inventario.**

Si el backlog tiene 27 HU y el sprint cubre 5, ¿cuáles 5?

Priorizar = decidir qué entregás primero para maximizar valor.



MoSCoW

Simple, rápido, el más común en cursos



WSJF

Cuantitativo, de SAFe, para backlogs grandes



Valor vs Esfuerzo

Visual, 2x2, ayuda a decidir quick wins

MoSCoW · Must / Should / Could / Won't

Aplicado al backlog ChangoMás.

Prioridad	Significado	Ejemplo
M · Must	No puede faltar en esta release; sin esto no hay producto	HU-011 Escanear · HU-015 Pagar tarjeta
S · Should	Importante pero puede postergarse un sprint	HU-016 Pagar QR · HU-020 Anular transacción
C · Could	Nice-to-have; si sobra tiempo	HU-050 Dashboard KPIs
W · Won't (esta vez)	Descartado para esta release	HU-052 Comparativo auto vs tradicional

⚠ **Regla de oro:** las Must no deben ser más del 60% del backlog inicial. Si son más, estás subestimando lo que se puede postergar.

WSJF · Weighted Shortest Job First

Cuando Must deja de discriminar · priorización cuantitativa (SAFe).

$$WSJF = \frac{Valor + Urgencia + Reducción\ de\ riesgo}{Tamaño\ (Story\ Points)}$$

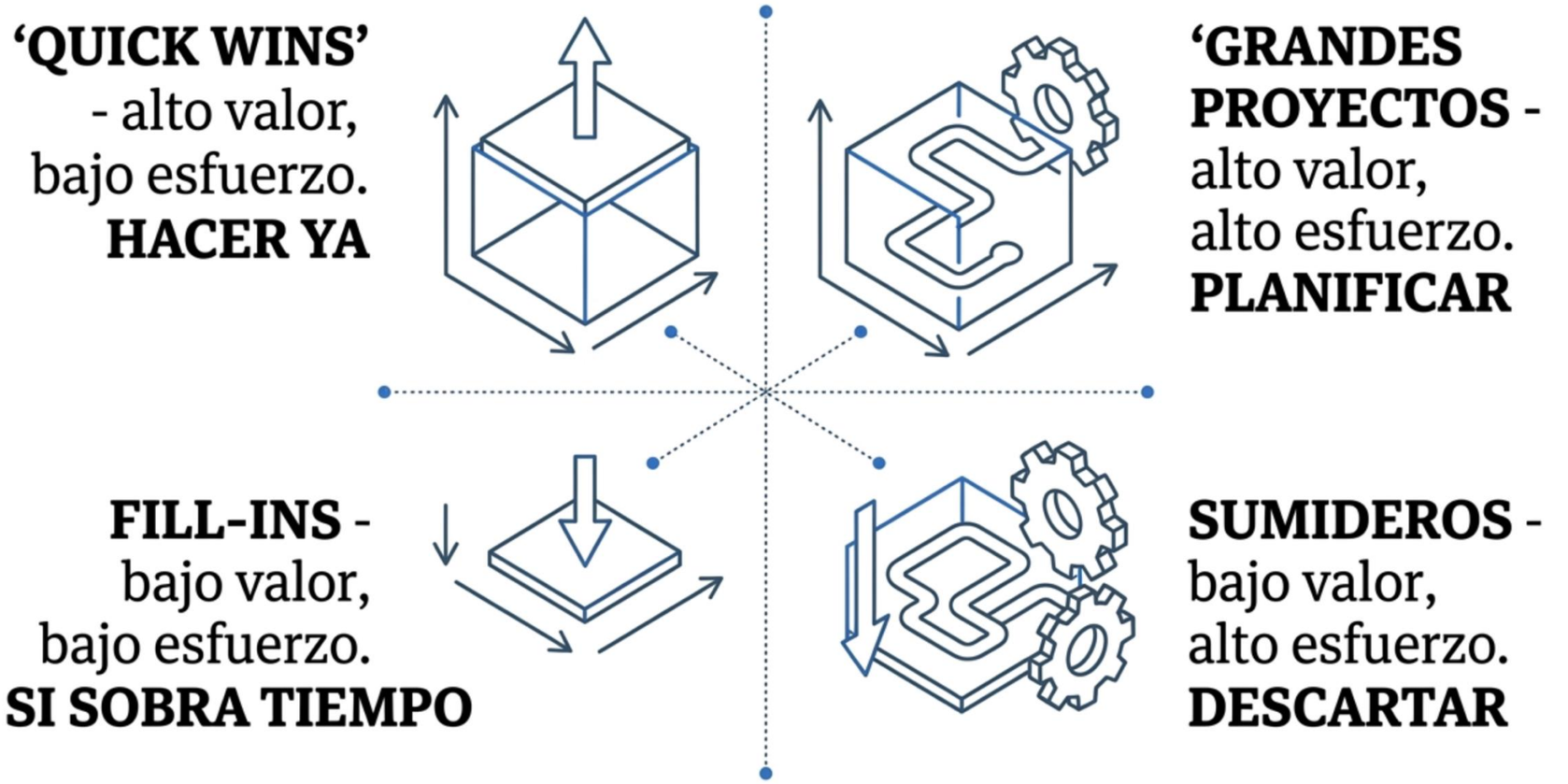
Se puntúa cada factor 1-10 en planning con el equipo.

HU	V	U	R	SP	WSJF
HU-011 Escanear	10	8	5	3	7.7
HU-021 Báscula anti-fraude	8	6	10	8	3.0
HU-050 Dashboard	5	3	2	8	1.2

Mayor WSJF = hacer primero.

Matriz Valor vs Esfuerzo

La herramienta visual más rápida para decidir.



Quick wins (alto valor + bajo esfuerzo) = HU ideales para empezar.

Sumideros (bajo valor + alto esfuerzo) = candidatos a Won't.

Estimación · ¿Por qué Story Points y no horas?

> **Relatividad y equipo · no burocracia.**

Estimar en HORAS — por qué falla

- ❌ Cada dev tarda distinto → hora de quién
- ❌ Factor tiempo real (interrupciones, contexto) impredecible
- ❌ Genera expectativa de compromiso en horas → se politiza
- ❌ Si uno dice 4h y otro 8h, discuten en vez de estimar

Estimar en STORY POINTS — por qué funciona

- ✅ Miden **complejidad relativa**, no tiempo absoluto
- ✅ Son del equipo, no del individuo
- ✅ Permiten planificación por velocity histórica
- ✅ Evitan debates sobre productividad individual

Escala Fibonacci · qué significa cada número

El salto entre números refleja incertidumbre creciente.

SP	Tamaño	Cómo reconocerlo
1	Trivial	Fix menor, cambio cosmético. <2h.
2	Pequeño	Task simple y clara. Medio día.
3	Medio-chico	Feature chica bien definida. 1 día.
5	Medio	Feature clara con algunos casos. 2-3 días.
8	Grande	HU completa compleja. Casi un sprint.
13	Partirla	Dudosa, muchos unknowns.
21+	Feature, no HU	Descomponer.

La distancia 5→8 es 60%, 8→13 es 60% · por eso Fibonacci.

Planning Poker · paso a paso

> **Estimación por consenso con cartas · no promedios.**

1. PO explica la HU al equipo (3 min)
2. Equipo pregunta hasta entender el alcance (2-5 min)
3. Cada dev levanta su carta simultáneamente
4. Si hay consenso → ese es el SP
5. Si hay dispersión → extremos explican su razonamiento
6. Se revota hasta consenso

⚠ **Reglas:** Nunca promediar — siempre buscar consenso. Timebox: 10 min máximo por HU.

Ejemplo · sesión real sobre HU-015

HU-015 · Como cliente quiero pagar con tarjeta de débito o crédito para cerrar mi compra sin pasar por un cajero.

Ronda 1 · dispersión alta

Dev	Estimación	Razonamiento
Dev A	3 SP	Ya integré una vez con Prisma, sé cómo es
Dev B	13 SP	Hay que integrar SDK + manejar errores + PCI DSS
Dev C	5 SP	Intermedio; sé de la API pero no de certificación

Discusión — Dev B argumenta que la certificación PCI DSS es no trivial.

5 SP

Resultado final

Ronda 2 · todos 5 SP. Consenso. HU-015 estimada.

Velocity del equipo y Capacity del sprint

La estimación sirve para planificar realísticamente.

Concepto	Qué mide	Cómo se calcula
VELOCITY	SP promedio que el equipo cierra por sprint	Histórico de últimos 3-5 sprints
CAPACITY	SP disponibles este sprint	Velocity × factor de disponibilidad
COMMITMENT	SP que el equipo se compromete	≤ Capacity

Ejemplo ChangoMás

18 SP	18 SP	16 SP
Velocity	Capacity S1	Commitment
Promedio últimos 3 sprints	Equipo completo, sin vacaciones	Deja 2 SP de margen para imprevistos

⚠ **Regla:** nunca comprometer más de la velocity histórica.

Ejercicio en clase · 20 minutos

> **Con tu backlog de Clase 8, aplicá todo lo de hoy.**

Consigna (en grupo):

1. Elegí 3 HU de tu backlog (5 min)
2. Validá INVEST una por una · marcá qué criterio falla si falla (5 min)
3. Escribí CA para la HU de mayor prioridad: 1 happy + 1 error + 1 borde (5 min)
4. Asigná prioridad MoSCoW a las 3 HU · justificá (3 min)
5. Jugá planning poker sobre 1 HU: cada integrante levanta carta (2 min)

Entregable: 1 hoja con 3 HU validadas + CA + prioridad + SP.

Tarea para casa · antes de Clase 10

Dejar el backlog Must listo para cargar en Azure DevOps.

📋 Calidad INVEST

Todas las HU Must cumplen los 6 criterios INVEST

📋 Criterios de Aceptación

Cada HU Must tiene al menos 3 CA (happy + error + borde)

📋 Estimación en SP

Cada HU Must estimada en Story Points

📋 Prioridad MoSCoW

Prioridad MoSCoW asignada a todas las HU del backlog

📋 Sprint 1 identificado

Identificadas las HU del Sprint 1 según capacity estimada

ⓘ **Total HU Must esperadas:** 8-12 HU · 25-40 SP (2 sprints de trabajo).

Próxima clase · Clase 10

SPRINTS, DOR/DOD Y TALLER AZURE BOARDS

Qué veremos:

01

Qué es un Sprint y cómo se elige la duración

02

Las 4 ceremonias: Planning, Daily, Review, Retro

03

Sprint Backlog vs Product Backlog

04

Descomposición de HU en Tasks

05

Definition of Ready (DoR) y Definition of Done (DoD)

06

🛠 Taller en vivo: cargar tu backlog en Azure Boards (40 min)

Arrancá Clase 10 con tu backlog Must **estimado y priorizado**.

Cierre · ¿Preguntas?

Lo que se lleva hoy:

1. INVEST

Una HU sin INVEST es un bug latente

2. CA

Los CA son el contrato con QA — sin ellos no hay Done

3. Priorizar

Priorizar es decidir qué NO hacer ahora

4. Story Points

Son relativos — no confundir con horas

5. Planning Poker

Consenso, no promedio

6. Velocity

Histórica > promesas optimistas